

The issue where non-default locales cause an "invisible" additional locale segment (e.g., `/ru/ru/check-in`) occurs because Next.js middleware and internal routing logic are both attempting to handle the locale prefix. [Stack Overflow +2](#)

Root Causes

- **Double Normalization:** By default, Next.js "normalizes" URLs by stripping the locale before middleware runs. If your middleware then manually adds a locale prefix to a rewrite, and Next.js's internal i18n routing also detects it, the final internal path becomes double-prefixed.
- **Middleware Path Rewriting:** When using `NextResponse.rewrite()`, the `x-middleware-rewrite` header reflects the internal destination. If the middleware prepends a locale (e.g., `/ru`) to a path that Next.js already identifies as being in the `ru` locale, the resulting internal path is `/ru/ru/...`. [HTTP.DEV +3](#)

Recommended Solutions

1. Enable `skipMiddlewareUrlNormalize`

Add this flag to your `next.config.js`. This prevents Next.js from stripping the locale from the `pathname` before it reaches your middleware, allowing you to manage the exact path structure yourself.

javascript



```
// next.config.js
module.exports = {
  i18n: {
    locales: ['en', 'ru'],
    defaultLocale: 'en',
  },
  skipMiddlewareUrlNormalize: true,
}
```

2. Use `next-intl` Middleware Standards

If using `next-intl`, ensure you are not manually rewriting paths in a way that conflicts with its `localePrefix` strategy. The library typically handles the `x-next-intl-locale` header to maintain the correct internal state without requiring manual path prepending.

3. Conditional Rewrite Logic

In your `middleware.js`, check if the `pathname` already starts with a valid locale before applying a rewrite. This prevents the "invisible" addition of a second locale segment.

javascript



```
// middleware.js example
const pathname = request.nextUrl.pathname;
const pathnameHasLocale = locales.some(
  (locale) => pathname.startsWith(`/${locale}/`) || pathname === `/${locale}`
);

if (!pathnameHasLocale) {
  const locale = getLocale(request);
  request.nextUrl.pathname = `/${locale}${pathname}`;
  return NextResponse.rewrite(request.nextUrl);
}
```

4. Verify Routing configuration

Ensure your `routing.ts` (if using `next-intl`) or `next.config.js` does not have conflicting `defaultLocale` settings that might trigger automatic redirects to a prefixed version of the default path. [GitHub +8](#)